

Weather Prediction ML — Phase 4 Report

Project Structure & Backend Setup

Muhammad Waleed Dar | AI/ML Internship

1. Overview

Phase 3 produced a trained Random Forest classifier (selected for its F1-score and cross-validation stability) predicting RainTomorrow on the WeatherAUS dataset, evaluated entirely inside a Jupyter notebook. This phase converts that notebook implementation into a modular, deployment-ready application: separated Python scripts, model serialization via Joblib, and a basic working Streamlit interface — satisfying the project structure, script separation, serialization, and basic-application requirements of the Phase 4 task sheet.

2. Tech Stack

Category	Tools / Libraries
Language	Python 3
Data Manipulation	Pandas, NumPy
ML & Preprocessing	Scikit-learn (LabelEncoder, MinMaxScaler, RandomForestClassifier)
Class Imbalance	Imbalanced-learn (SMOTE-Tomek)
Model Serialization	Joblib
Application Interface	Streamlit
Environment	Virtual environment (venv), VS Code
Version Control	Git & GitHub

3. Project Structure

The notebook implementation was split into single-responsibility scripts, matching the task sheet's suggested breakdown (preprocessing, feature engineering, model loading, prediction pipeline, utilities, main application):

```
Weather-Prediction-ML/
|-- data/weatherAUS.csv
|-- models/                  (random_forest_model.joblib, scaler.joblib,
|                             encoders.joblib, feature_columns.joblib)
|-- src/
|   |-- utils.py             (shared feature list, threshold, paths)
|   |-- preprocessing.py     (cleaning: nulls, duplicates, outliers)
|   |-- feature_engineering.py (derived features + shared categorical encoding)
|   |-- model_loader.py      (loads serialized artifacts)
|   |-- predict.py           (inference pipeline)
|   |-- train_model.py       (orchestrates the full pipeline, saves artifacts)
|-- app/streamlit_app.py     (Streamlit interface)
```

```
`-- requirements.txt
```

Interface choice: Streamlit was selected over FastAPI/Flask because the Phase 4/5 deliverable is a directly-usable visual demo for a live presentation, not an API consumed by another service — Streamlit allows the interface to be built in pure Python without separate HTML/CSS/JavaScript.

4. Issues Faced & How They Were Resolved

4.1 Rain-class recall was low on the default classification threshold

Issue: Mentor feedback on the Phase 3 submission flagged that the model was under-predicting the Rain class. Investigation showed the pipeline already used SMOTE-Tomek (training set only) and MinMax scaling — the test set was intentionally left at its natural ~84/16 imbalance, so raw predictions skewed toward "No Rain" as expected. The real, addressable issue was Random Forest's Recall of 55.95% on the Rain class at the default 0.5 threshold.

Resolution: Added a threshold-tuning step: evaluated Recall/Precision/F1 across thresholds 0.5 down to 0.25 using the model's predicted probabilities. A threshold of 0.35 was selected, raising Rain Recall to 76.47% (Precision dropped from 58.04% to 43.47%), accepted because missing an actual rain day is costlier than a false alarm for this use case.

4.2 Feature order / encoding mismatch risk between training and inference

Issue: The trained model, scaler, and label encoders are all order- and value-sensitive. Building the prediction script independently from the training script risked the two drifting apart — e.g. a different column order, or the UI offering a categorical value an encoder was never trained on.

Resolution: Centralized the feature list and categorical column list in a single `utils.py` module imported by both scripts. Built a shared `encode_categoricals()` function in `feature_engineering.py` used in "fit" mode during training and "transform" mode during inference, and added an explicit check in the prediction pipeline that raises a clear error if a submitted category was never seen during training, instead of failing silently or producing an incorrect prediction.

4.3 Null-imputation logic silently failed to catch text columns on newer pandas versions

Issue: The original cleaning script checked `df[col].dtype == "object"` to decide whether to impute a column with its mode (categorical) or median (numeric). When tested end-to-end, this raised a `TypeError` attempting to compute a median on a text column.

Resolution: Root cause: pandas 3.0+ uses a dedicated "str" dtype for text columns instead of "object", which the equality check no longer matched. Replaced the check with `pandas.api.types.is_numeric_dtype()`, which correctly identifies numeric columns regardless of the pandas version installed, making the preprocessing script robust across environments.

4.4 requirements.txt version pinning

Issue: Hardcoding specific package version numbers risked listing versions that don't match what's actually installed in the working environment, which can cause install conflicts for anyone else setting up the project.

Resolution: `requirements.txt` lists only the required package names; the actual pinned versions are generated locally via ``pip freeze`` after installation, ensuring the committed file reflects a real, working environment rather than guessed version numbers.

5. Status & Next Steps

Phase 4 deliverables — modular scripts, model serialization, and a basic working Streamlit interface — are complete and ready for repository submission, pending a final training run against the real weatherAUS.csv dataset to generate the committed model artifacts. Phase 5 will extend this into the full application (refined interface, deployment) due August 11, 2026.